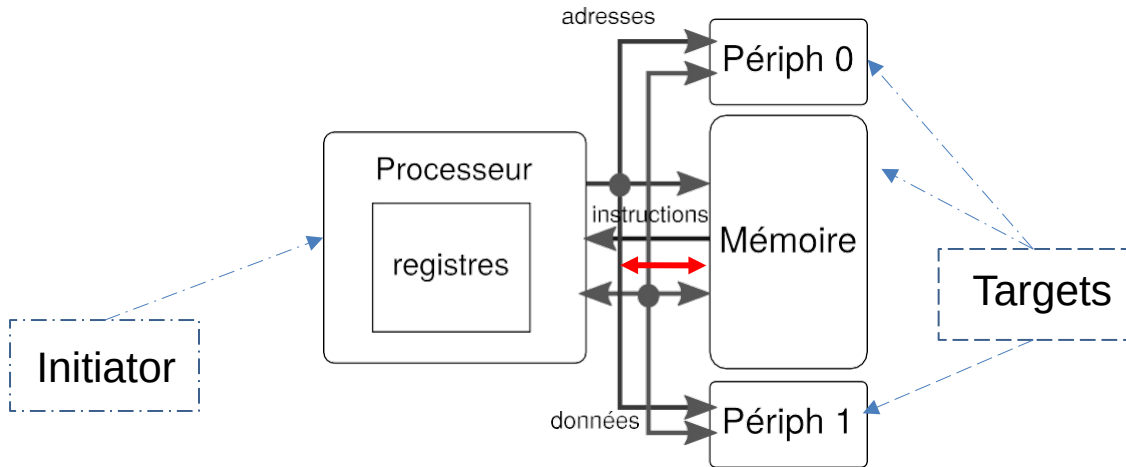


SoC : Systèmes d'interconnexion

Pour les SoC actuels, les périphériques sont **entrelacés** en mémoire

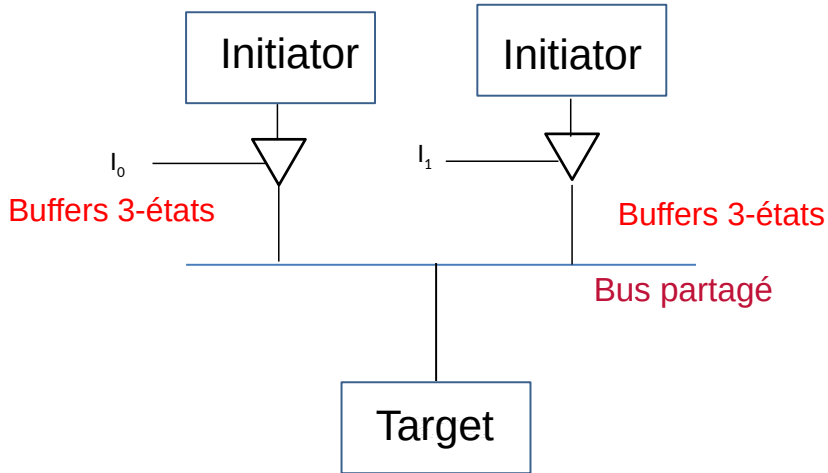


Le mappage mémoire

- Périphériques différenciés en fonction de l'adresse
- Simplification de la conception des processeurs
- Le processeur (Maître) initie des transactions (de lecture/écriture)
- Les périphériques et la mémoire sont des cibles répondant à ces transactions
- D'autres éléments peuvent initier des transactions : contrôleur DMA (*direct Memory Access*), coprocesseur, accélérateur...

Question : comment est implémenté ce mappage mémoire sur un système sur puce

Bus partagé



Bus partagé ..

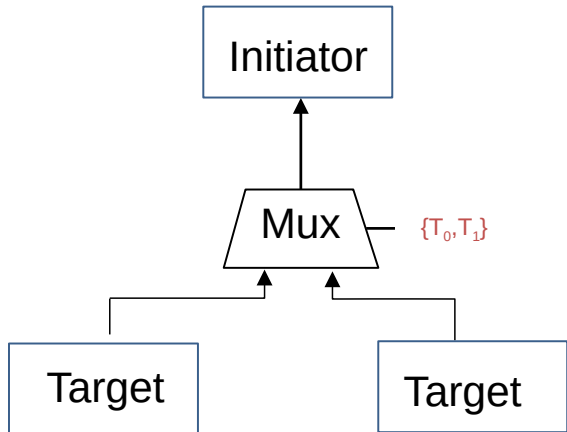
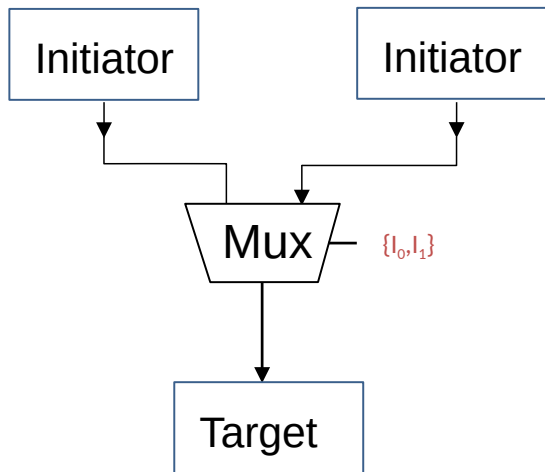
- Bus partagé en utilisant un buffer 3 états
- Initiateur et cible sélectionnés sont les seuls connectés, les autres sont mis en état haute impédance
- Si plusieurs initiateurs, besoin d'un arbitre pour contrôler les accès

Problèmes:

- La charge électrique change en fonction du nombre de cibles : **temps de propagation non garanti**
- **Inadéquation** par rapport à la densité d'intégration offerte par les Socs (dans un SoC avec des dizaines d'IP, un bus partagé physique imposerait des fils longs et chargés traversant tout le chip, ce qui augmente la capacité parasite et limite la fréquence).

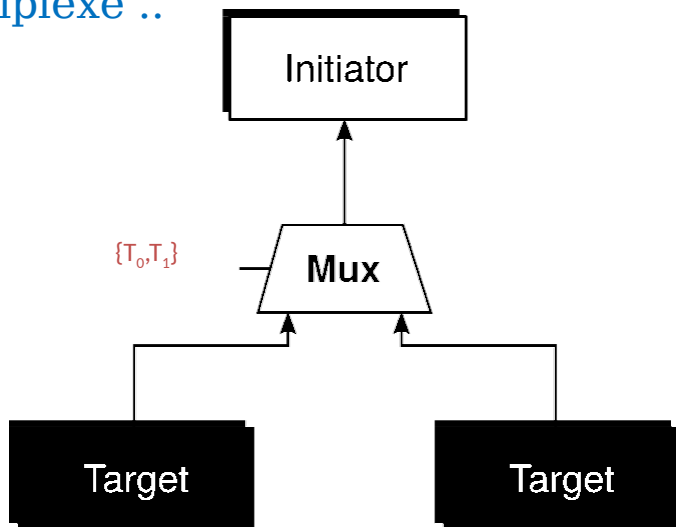
Ce type de bus est plutôt réservé à des bus externes (I2C, SPI, ...)

Bus multiplexé

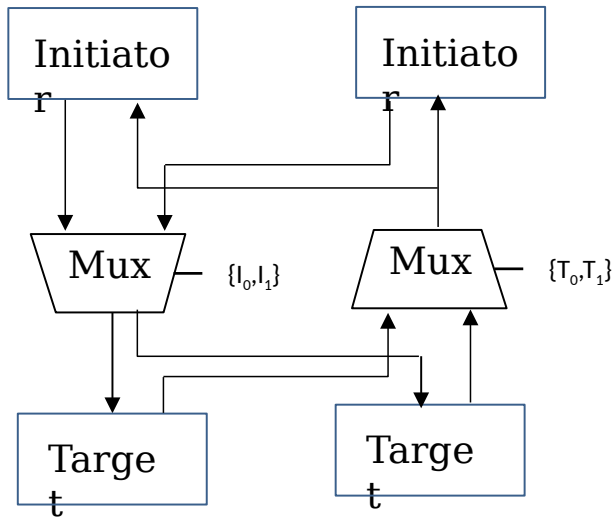


Logique de contrôle pilotant I_0, I_1 T_0, T_1

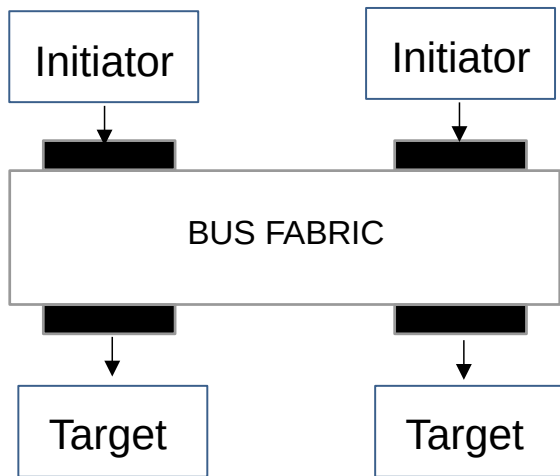
Bus Multiplexé ..



Bus bidirectionnel multiplexé



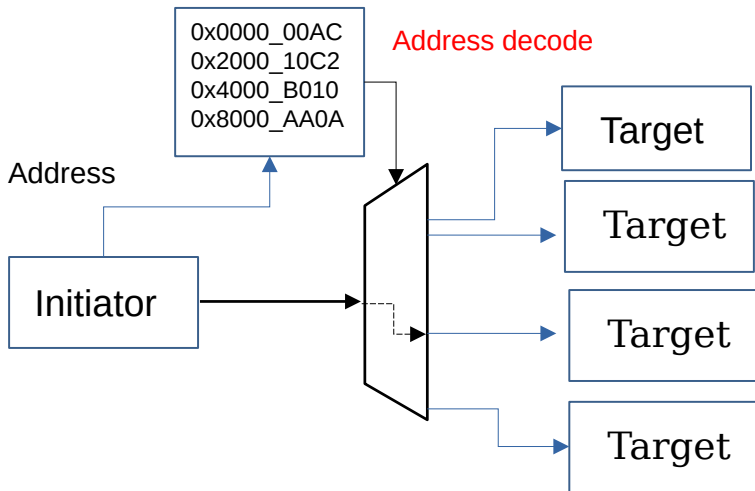
Bus fabric



Bus fabric :
système d'interconnexion
intégrant une logique de
contrôle, un système de
multiplexage, des registres et
des files d'attente

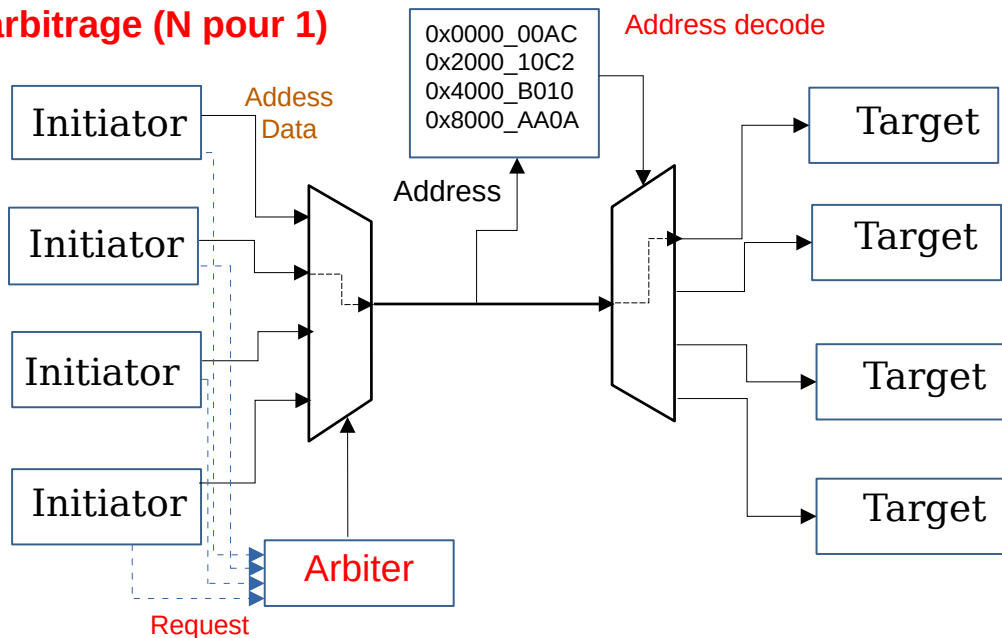
Fonctionnalités de base d'un système d'interconnexion

L'Adressage



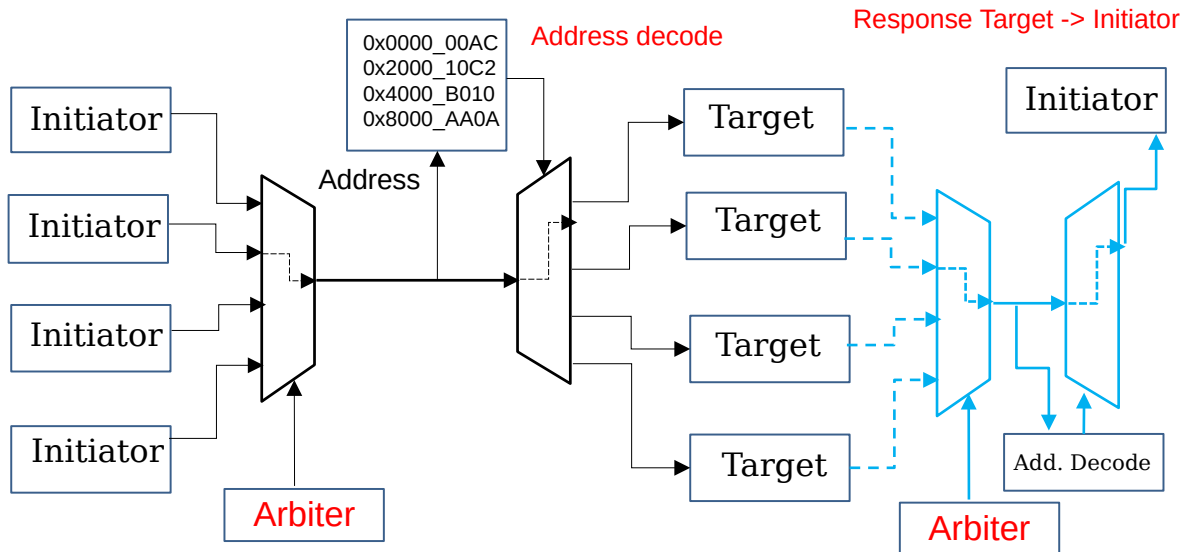
Fonctionnalités de base d'un système d'interconnexion

L'arbitrage (N pour 1)

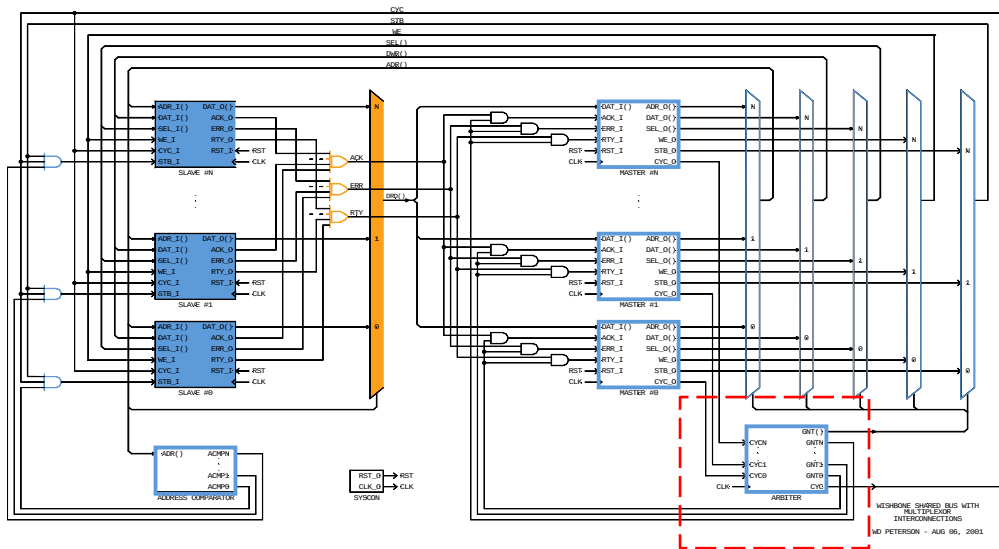


Fonctionnalités de base d'un système d'interconnexion

L'arbitrage : chemin de retour



Exemple de bus multiplexé : Wishbone



Arbitrage, décodeur d'adresses et multiplexage.

A cette architecture d'interconnexion physique doit être adjoint la définition des mécanismes d'échanges

Protocoles de communication sur puce

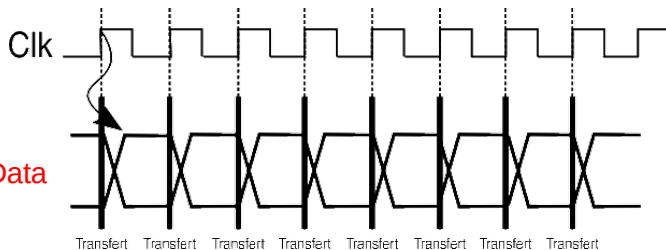
Protocole

- Définit les transactions entre une cible et un initiateur
- Indépendance par rapport à la topologie du système d'interconnexion
- Définit l'interfaçage avec le système d'interconnexion
- Utilisable dans un contexte de périphériques mappés en mémoire
- Utilisable pour différents niveaux de performances
- Permet des communications à faibles latences et/ou à haut débits

Une standardisation garantit l'interopérabilité

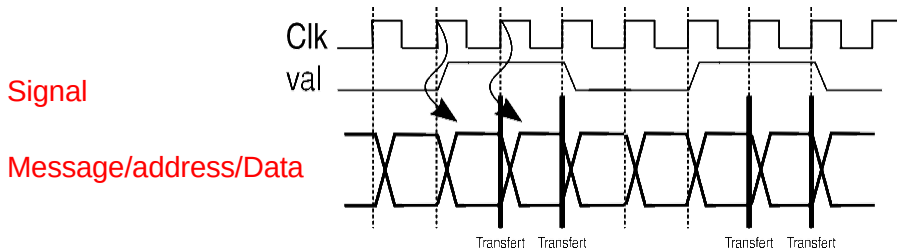
Protocoles synchrones

- Nécessité d'une horloge
- Garantie de performance
- Les sorties des initiateurs changent au front d'horloge
- Les cibles capturent au front d'horloge



Signalisation

Signal -> information additionnelle pour la validation d'un transfert



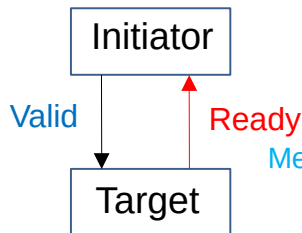
Protocole synchrone-signalisation

L'initiateur peut signaler les cycles pour lesquels un transfert est valide

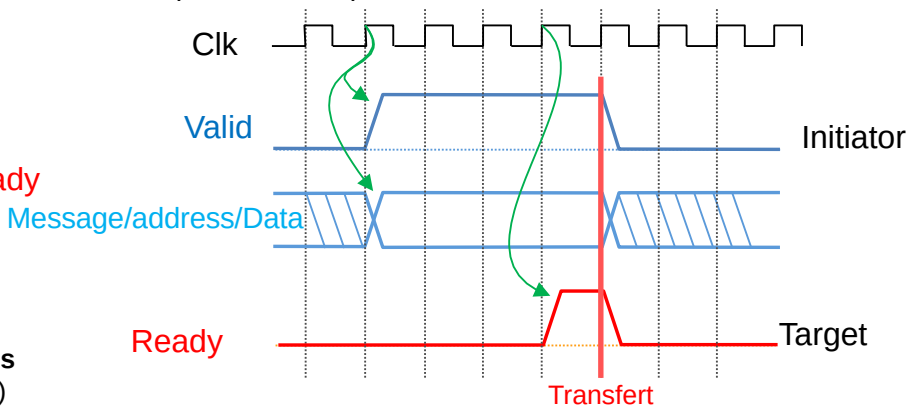
Rendez-vous

Mécanisme du rendez-vous (*handshake*) :

- Signalisation
- Un transfert n'a lieu :
 - s'il est validé par un initiateur
 - si la cible est prête à l'accepter

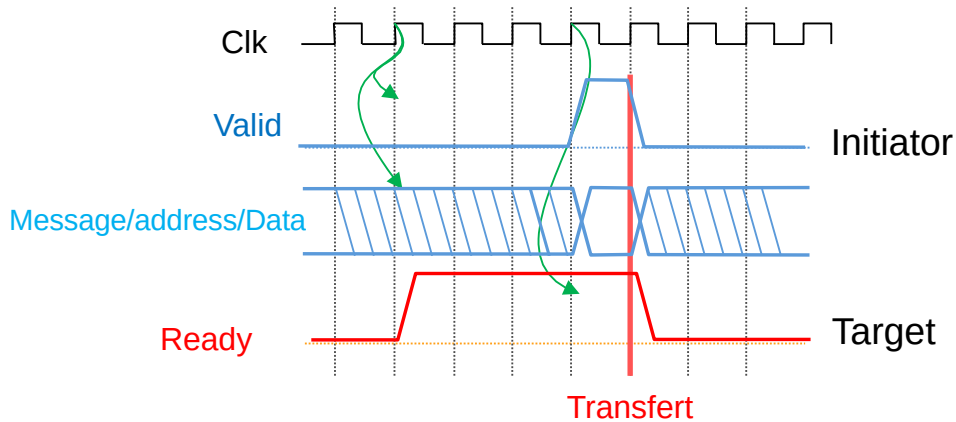


Rendez-vous
(*handshake*)



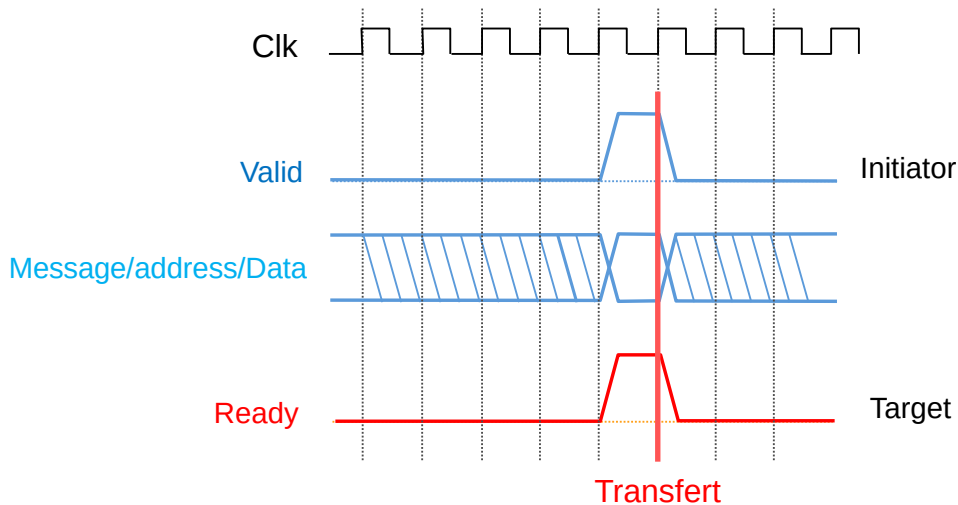
La cible met en attente l'initiateur : **Back pressure**

Rendez-vous ...



La cible peut être **prête à l'avance**

Rendez-vous ...

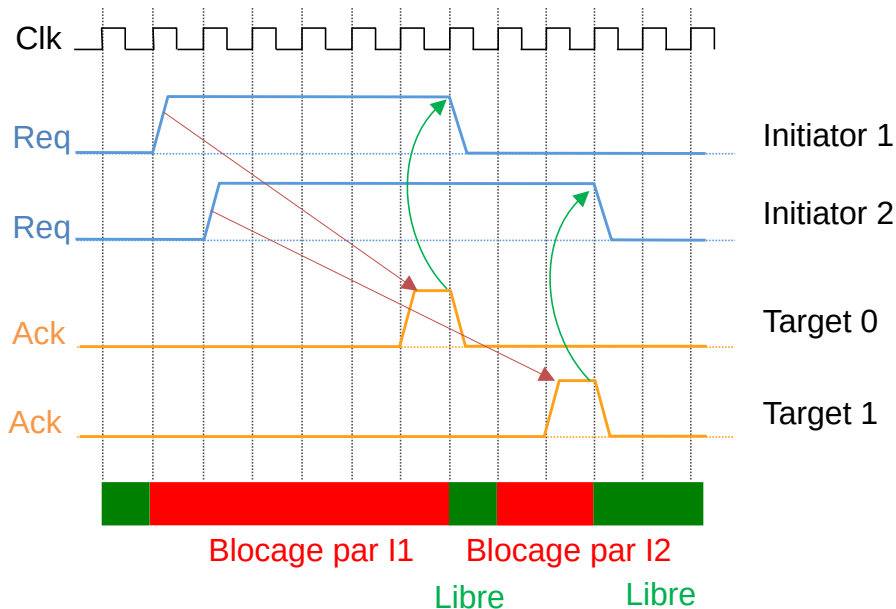


Cible et initiateur prêts **pendant** le même cycle

Transactions bloquantes

- Certains protocoles peuvent être bloquants
- Requête initiée et non finalisée : les autres initiateurs n'ont pas accès à la ressource partagée
- Pénalisant avec des **cibles avec une latence importante**

Transactions bloquantes ...



Transactions bloquantes ...

Séparation des requêtes/réponses

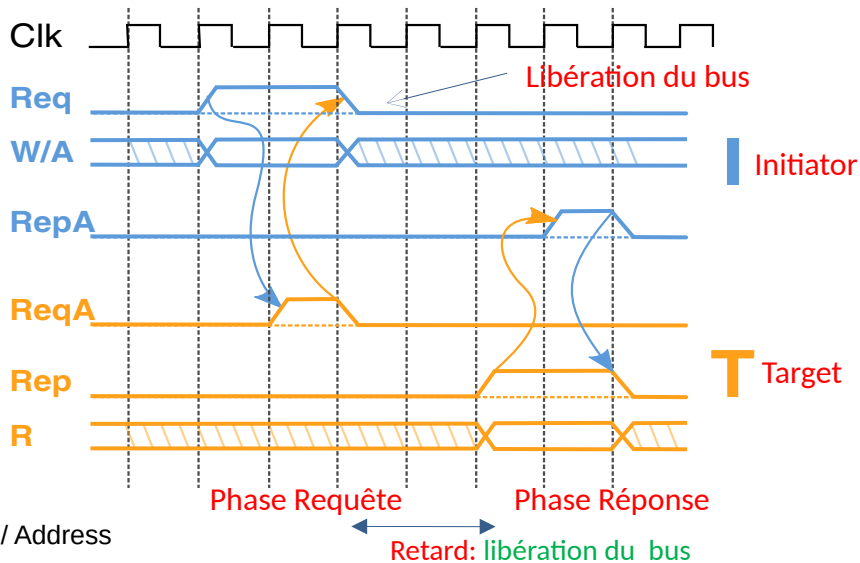


Transaction décomposée :

- Plusieurs phases.
 - de l'initiateur vers la cible (Req)
 - la cible vers l'initiateur (Ack)
- Libère la ressource partagée durant la préparation de la réponse de la cible.
- Taux d'utilisation du bus partagé est amélioré.
- Complexité accrue du protocole et les ressource nécessaire : deux rendez-vous distincts.

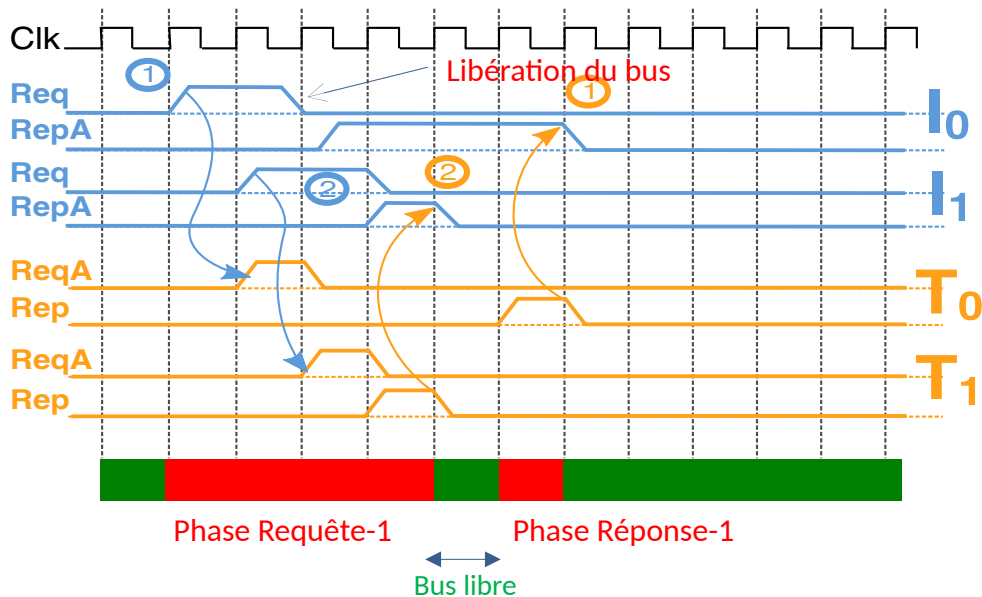
Transactions bloquantes ...

Séparation des requêtes/réponses -> limitation pénalité blocage bus



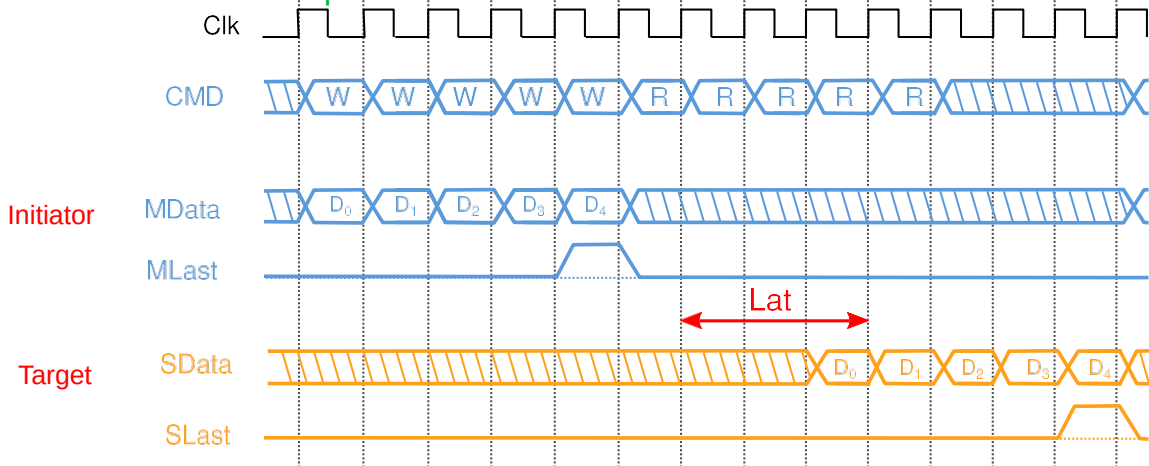
Transactions bloquantes ...

Séparation des requêtes/réponses -> limitation pénalité blocage bus



Transfert multiples (mode burst)

Transactions multiples

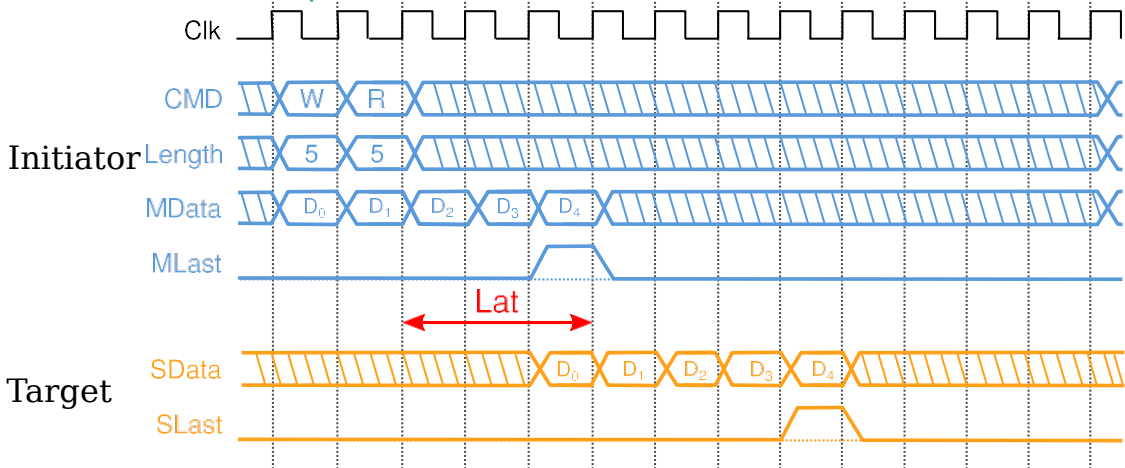


- L'initiateur écrit 5 données consécutives, puis lit 5 données
- 5 requêtes d'écriture suivies des 5 requêtes de lecture

MRMD: *Multiple Requests for Multiple Data transfers*

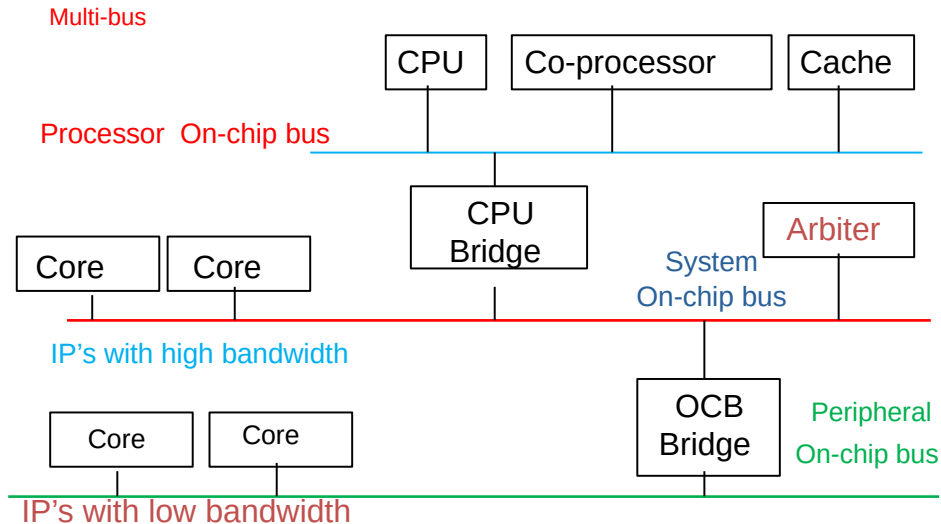
Transfert multiples (mode burst) ...

Transactions uniques



SRMD : Single Request for Multiple Data transfers

Rappel : Architecture SoC typique. Stratification de bus



OCB : On Chip Bus

Comparatifs de différents protocoles

	rdv. synch	Req/Resp	SRMD
ARM AXI	+	+	+
ARM AXI-Lite	+	+	-
OpenCore OCP	+	+	+
ARM AHB/APB	+	-	+
Whishbone	+	-	-
Altera Avalon	+	pipeline	burst ext.

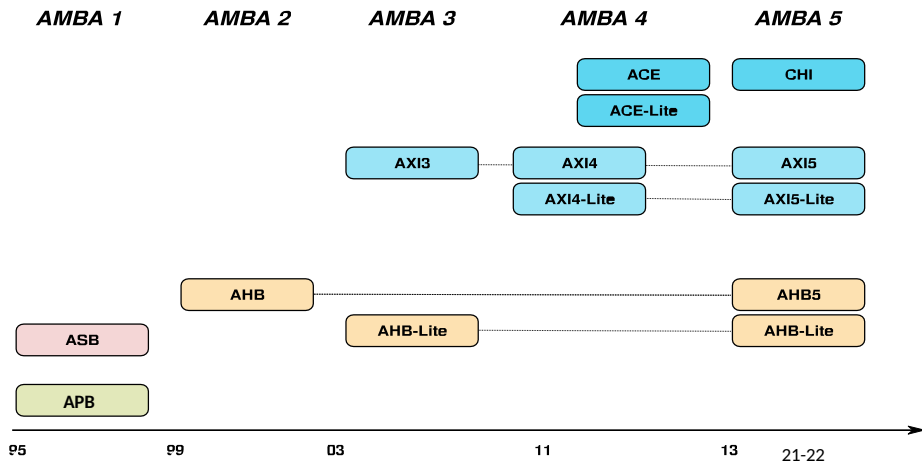
Pour avalon, SRMD n'est pas natif (extension)

Bus AMBA

AMBA: Advanced Microcontroller Bus
Architecture

AMBA: Advanced Microcontroller Bus Architecture

- Initialement propriétaire ARM,
- Devenu un standard de fait
- Plusieurs évolutions pour répondre au besoin des SoC modernes



Déclinaisons des divers protocoles AMBA

- ASB** : Advanced System Bus (**Bus bidirectionnel**)
- APB** : Advanced Peripheral Bus (**Périphériques lents**)
- AHB** : Advanced High Performance Bus (**Uni directionnel - Burst multi requêtes**)
- AXI** : Advanced eXtensible Interface
(Burst mono requête, **Séparation requêtes/Réponses en canaux**)
- ACE** : **AXI Coherency Extension** (Gestion de la cohérence de cache)
- CHI** : **Coherent Hub Interface** (Évolution gestion de la cohérence de cache)

Comparatifs

Caractéristique	AXI	AHB	APB
Performance	Très haute performance	Performance intermédiaire	Faible performance
Latence	Faible	Moyenne	Haute
Bande passante	Haute	Moyenne	Faible
Transactions simultanées	Oui	Non	Non
Flexibilité	Très flexible	Moins flexible	Très simple et fixe
Applications	Processeurs multicœurs, GPU, systèmes haute performance	Systèmes à performance intermédiaire	Périphériques lents et simples

Comparatifs ...

Caractéristique	AXI	AHB	APB
Type de transaction	Transactions en parallèle et série	Transactions synchrones, séquencées	Transactions simples et synchrones
Canaux de communication	Multiples canaux indépendants (lecture, écriture, adresse)	Un canal de commande, pas de parallélisme	Un canal pour toute la communication
Réponses	Réponses indépendantes (out-of-order)	Réponse unique pour chaque transaction	Réponse simple
Flexibilité	Très flexible, hautement configurable	Moins flexible que AXI, mais plus performant que APB	Très simple, dédié aux périphériques lents
Utilisation typique	Systèmes haute performance (multicœurs, GPU)	Systèmes embarqués à performance intermédiaire	Périphériques à faible vitesse (GPIO, UART)

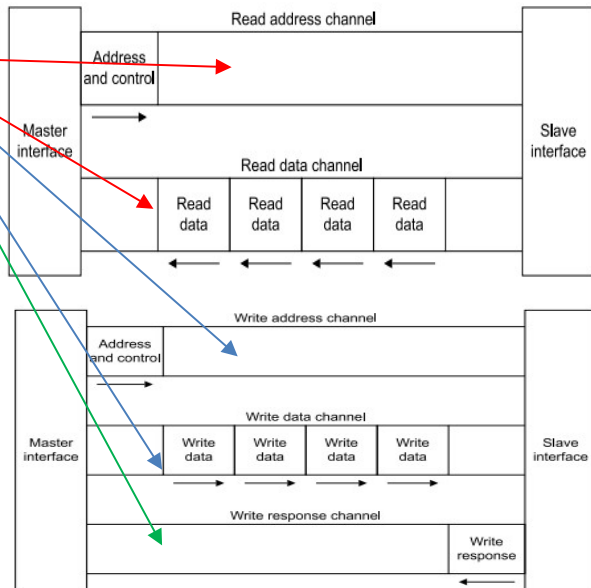
Le protocole AXI

AXI: Advanced eXtensible Interface

Caractéristiques

5 canaux

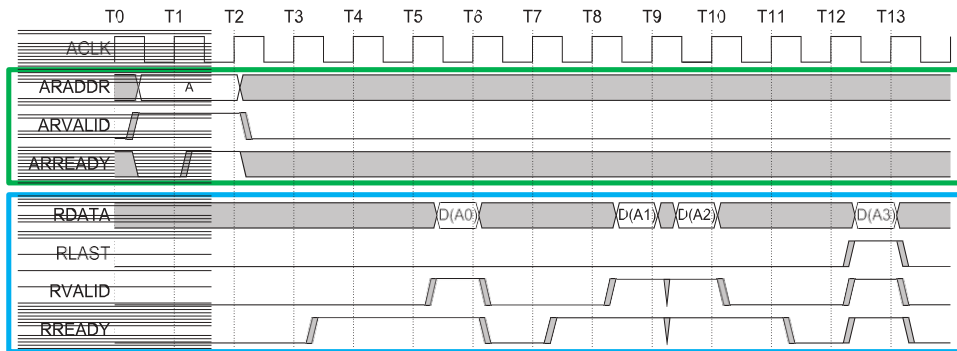
- Lecture et écriture séparées
- Canaux dédiés
- Requêtes et réponses séparées
- Conçu pour des SoC hétérogènes
- Forts besoins en débit
 - Accès multiples (burst)
 - Multi-maîtres (Plusieurs maîtres/threads)
- Mode spéciaux (permission, caches...)



Exemple de lecture de données

Maitre
(Initiator)

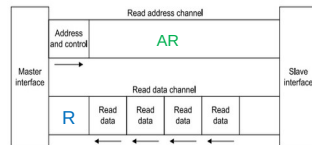
Esclave
(Target)



Extrait de AMBA AXI Protocol v2.0

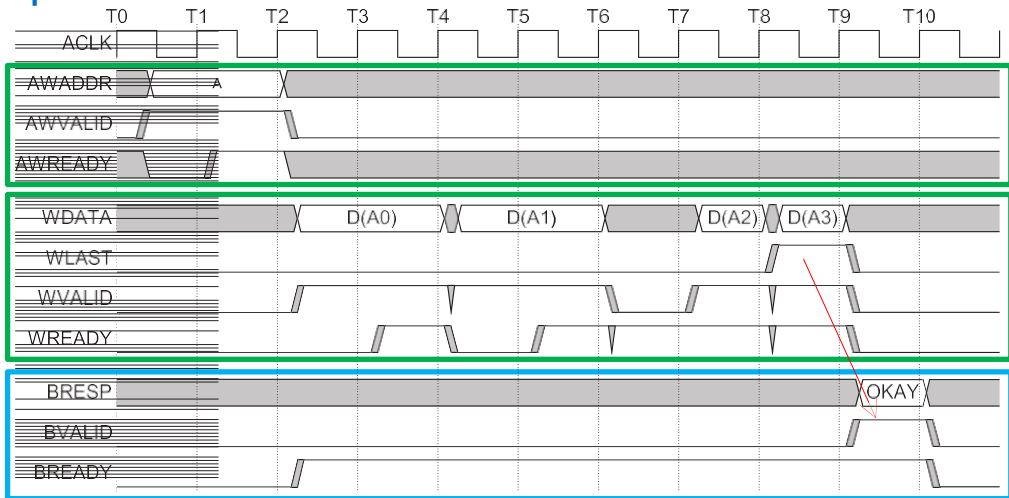
AR : Read address channel

R : Read data channel



Exemple d'écriture de données

Write
address channel

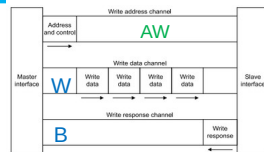


Extrait de AMBA AXI Protocol v2.0

AW : Write address channel

W : write data channel

B : Write response channel



Le Bus Avalon

Caractéristiques

- Développé par Altera (Intel) pour des SoC sur FPGA
- Pour des systèmes Memory Mapped
- Des extensions pour le Pipeline et le mode Burst
- Utilisé principalement pour les IPs Altera et le cœur de processeur Nios-II/Nios V
- **Les signaux de contrôle ne sont pas tous obligatoires**

Protocole synchrone

- Un cycle est défini d'un **front montant** à l'autre de l'horloge système.
- Le protocole est synchrone :
 - Tous les transferts commencent au front montant,
 - Tous les signaux sont donc générés par rapport à Clk,
 - Les signaux doivent être stables pendant l'état haut (hold-time),
- Il est possible de connecter des périphériques asynchrones (comme des mémoires off-chip : clk différent).

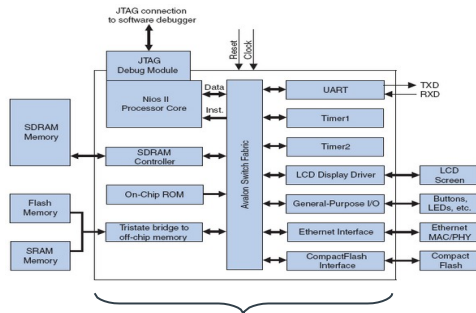
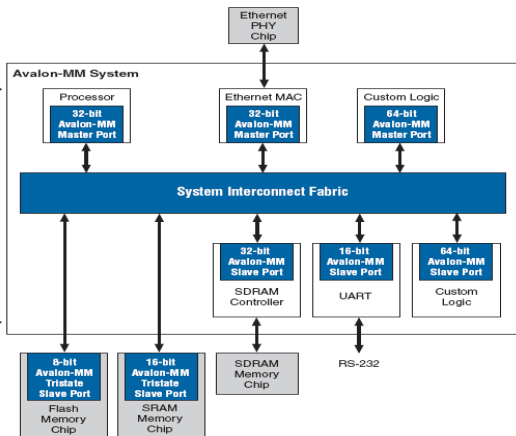
Dans ce dernier cas, le concepteur doit faire en sorte que les signaux soient stables pendant l'état haut de clk(**logique complémentaire**)

Plusieurs modes

- Les interfaces Avalon supportent les propriétés suivantes :
 - Wait-states : insérés par les esclaves (fixe ou variable),
 - Pipeline (envoie plusieurs requêtes sans attente de la réponse de la précédente
 - amélioration du débit),
 - Burst (transfert un bloc de données consécutives en une seule transaction
 - efficace pour les transferts mémoire),
 - Tristate (plusieurs composants de partagent le même bus - un seul parle à la fois),
 - Flow-control (Mécanisme de régulation du flux de données entre émetteur et récepteur)
- Le mode fondamental n'utilise aucune des propriétés ci-dessus.

Exemple de système basé sur le bus Avalon

Interne à la puce



Interne à la puce

6 types d'interfaces

- configurables par des propriétés et des paramètres spécifiques

Avalon-MM : liaison permettant des opérations lecture/écriture basée sur des adresses en mode maître/esclave pour des éléments internes au FPGA

Avalon-MM Tristate : liaison permettant des opérations lecture/écriture basée sur des adresses en mode maître/esclave pour des éléments externes partageant les bus

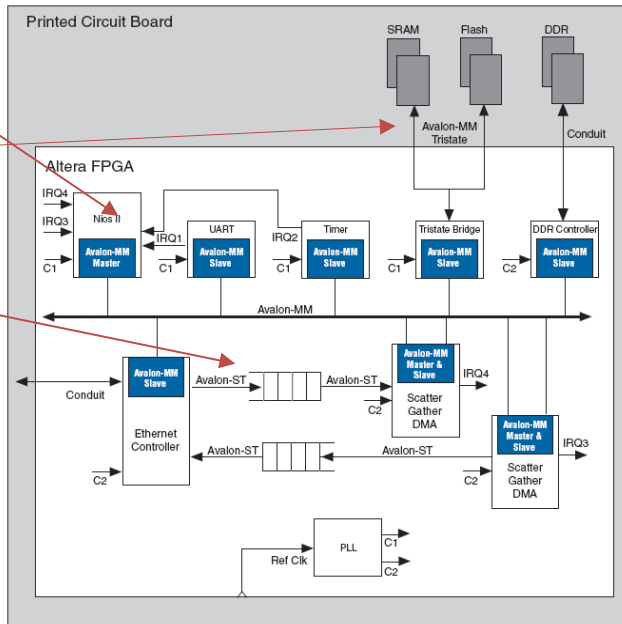
Avalon-ST : liaison permettant des envois de données **unidirectionnels** éventuellement multiplexés

Blocs d'interfaces dédiés

Avalon-clock : liaison permettant de recevoir ou d'envoyer des signaux d'horloge et de reset pour synchroniser les éléments du système

Avalon-interrupt : liaison permettant à des composants de signaler des événements

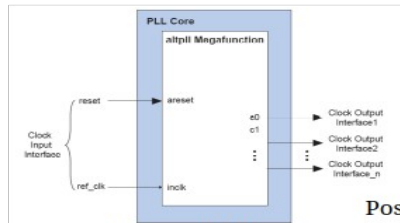
Avalon-conduit : liaison permettant des accès directs aux composants interne d'un système Qsys



Interfaces utilisables avec des systèmes à base de NIOS II

Interface horloge

Avalon-clock : signaux d'horloge et de reset (asynchrone)



Exemple d'une
PLL

Pas de propriétés
associées

Signaux d'entrée (clock sink):

Signal Type	Width	Direction	Required	Description
clk	1	Input	No	A clock signal. Provides synchronization for internal logic and for other interfaces.
reset	1	Input	No	Reset input. Resets the internal logic of an interface or component to a determined state.
reset_n				reset is synchronized to the clock input in the same interface.

Possibilité d'associer une horloge à d'autres entrées (associated clock)



Les resets sont tous reliés au reset global

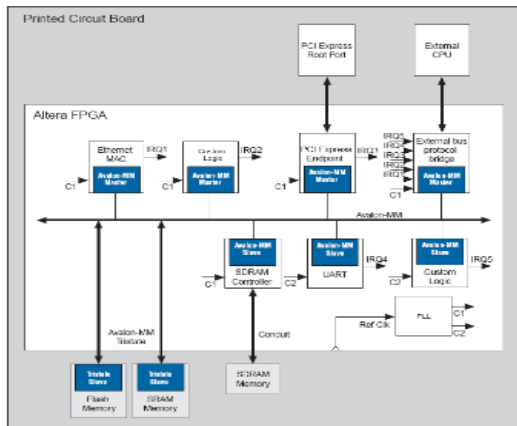
Signaux de sortie (clock source):

Signal Type	Width	Direction	Required	Description
clk	1	Output	Yes	An output clock signal.

Interfaces utilisables avec des systèmes à base de NIOS II

Interface Avalon MM

Avalon-MM : liaison permettant des échanges entre un ou plusieurs maîtres et un ou plusieurs esclaves à travers un système d'interconnexions spécifique



3 types d'interfaces

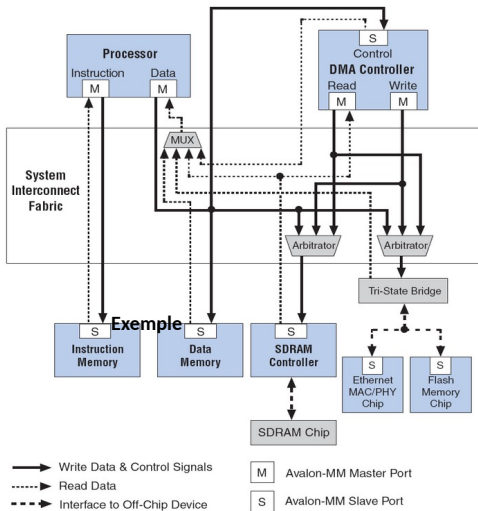
Avalon MM Slave

Avalon MM Master

Avalon MM Tristate Slave

Exemple d'un système comportant les trois interfaces

Principe du réseau d'interconnexions Avalon MM



Permet de relier un nombre quelconque d'éléments maîtres et d'éléments esclave

Composants au sein du FPGA: pas de ressources partagées - plusieurs bus en parallèle avec sélection par multiplexeur

Possibilité d'interfacer des éléments extérieurs au FPGA

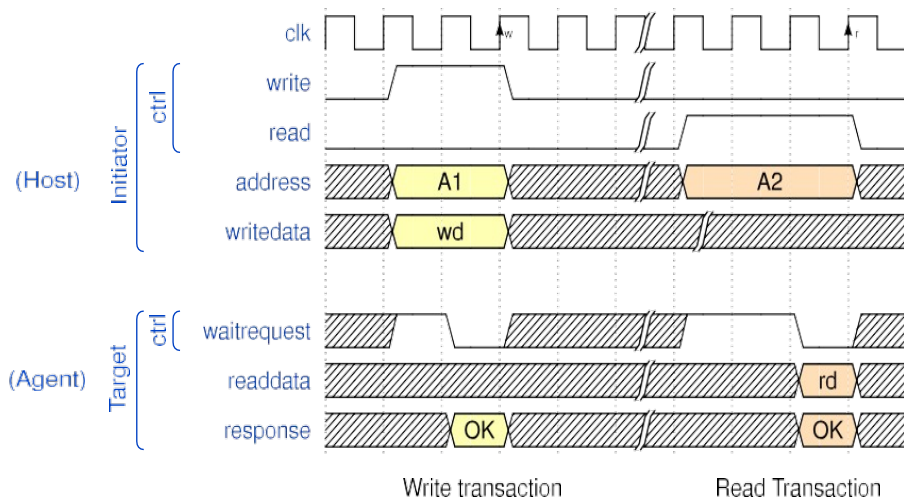
Composants peuvent utiliser différentes horloges

Composants maître/esclave peuvent avoir des tailles de données différentes

Composants peuvent avoir plusieurs ports

Généré automatiquement par Qsys

Avalon: Transactions simples



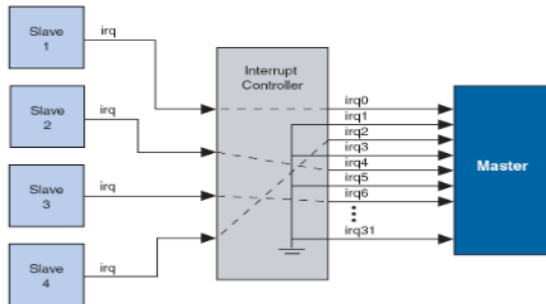
Interfaces utilisables avec des systèmes à base de NIOS II

Principe du réseau d'interconnexions spécifiques Avalon MM

Gestion des interruptions :

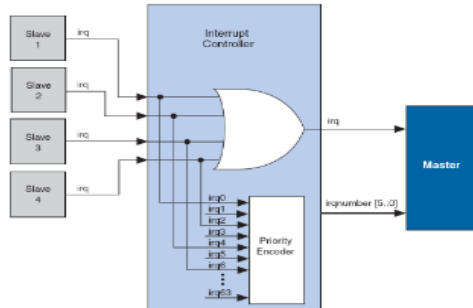
Si les ports esclave génèrent des demandes d'interruption, des contrôleurs d'interruption sont insérés pour chaque port maître qui accepte les interruptions.

Deux types de contrôleurs sont possibles



Priorité Soft

Use	C...	Name	D...	Export	Clock	Base	End	IRQ
☒		External_Clocks	clk		clk			
☒	R	clk	clk...		Exter...	0x0000_0000	0x0000_07ff	
☒	R	CPU	No...		Exter...	0x1000_0000	0x1000_002f	
☒	R	axiadm	Sy...		Exter...	0x0000_0000	0x01ff_ffff	
☒	R	SDRAM	SD...		Exter...	0x1000_1000	0x1000_100f	
☒	R	JTAG_UART	JT...		Exter...	0x1000_2000	0x1000_201f	
☒	R	Interval_timer	Inte...		Exter...	0x0200_0000	0x0200_000f	
☒	R	IR_data	Per...		Exter...	0x0200_0010	0x0200_001f	
☒	R	commande_moteurs	Per...		Exter...	0x0200_0020	0x0200_002f	
☒	R	sensors0	Per...		Exter...	0x0200_0030	0x0200_003f	
☒	R	sensors1	Per...		Exter...	0x0200_0040	0x0200_004f	
☒	R	timer_rotat	cle...		Exter...	0x0200_0060	0x0200_006f	
☒	R	automate_spi_accelerom...	aut...		Exter...			



Priorité Hard

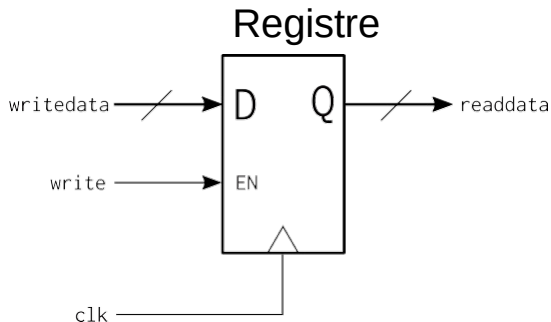
Exemple pour un simple registre :

Les signaux de contrôle ne sont pas tous obligatoires.

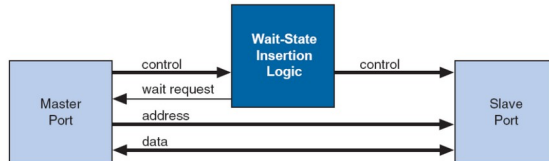
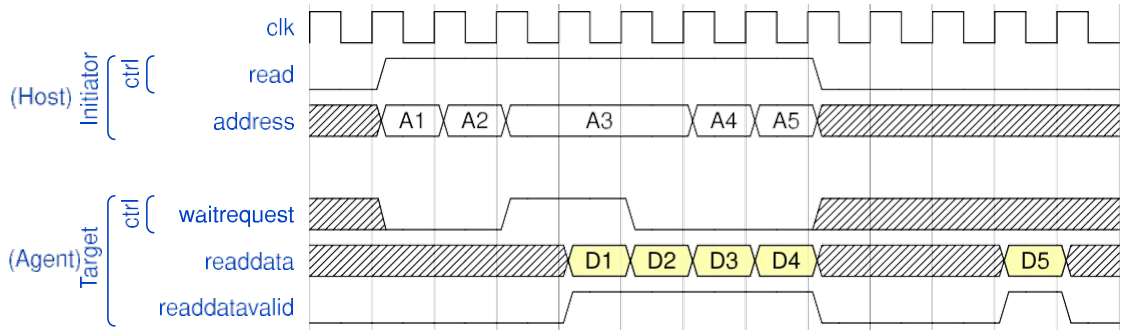
Exemple :

Les données en sortie d'un registre sont toujours prêtes.

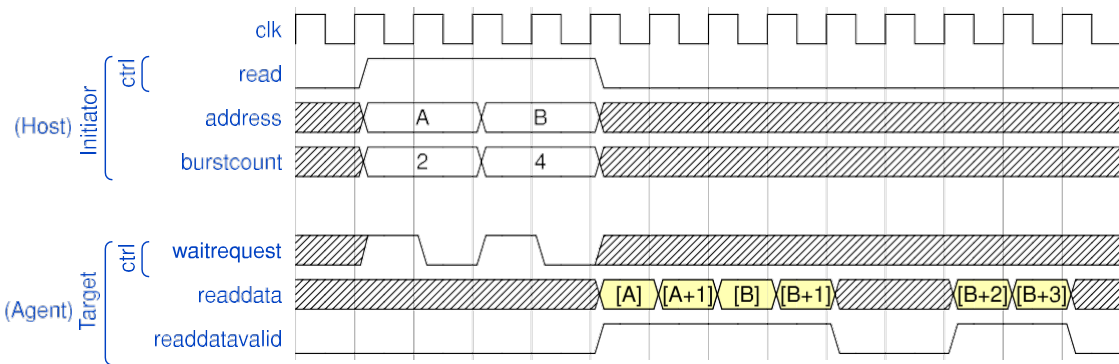
Les signaux *waitrequest* et *read* ne sont pas utiles



Avalon : lecture mode pipeline (extension)



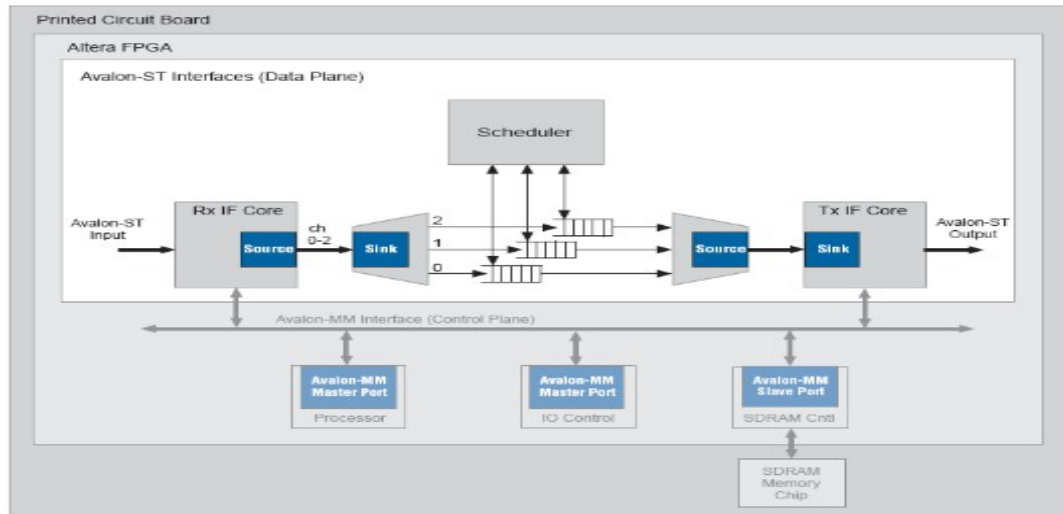
Avalon : lecture mode burst (extension)



Interfaces utilisables avec des systèmes à base de NIOS II

Interface Avalon ST

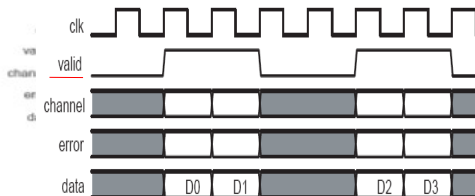
**Liaisons unidirectionnelles à grande bande passante et faible latence
(transmission de paquets éventuellement entremêlés)**



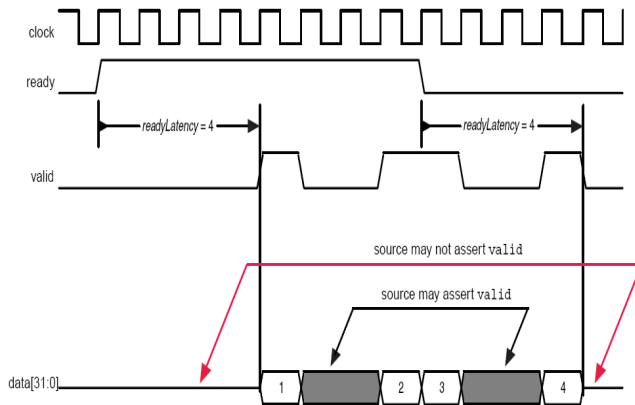
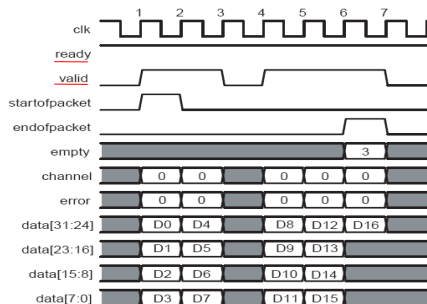
Interfaces utilisables avec des systèmes à base de NIOS II

Interface Avalon ST – transferts

Transfert sans backpressure



Transfert de paquets



Il n'y a pas d'adresse : les données coulent dans un seul sens.